

Problemática 2: Paralelización con OpenMP para el Análisis de Datos IMU

Aldo Martínez Cruz
Facultad de Ciencias, UNAM

17 de noviembre de 2025

1. Introducción

El análisis de la marcha humana mediante Unidades de Medición Inercial (IMU) genera grandes volúmenes de datos temporales. Un preprocesamiento eficiente de estos datos es crucial para las tareas posteriores de análisis.

El objetivo de este trabajo es implementar un programa de cómputo paralelo utilizando **OpenMP** para acelerar el cálculo de métricas estadísticas sobre estos datos, empleando un enfoque de paralelización por ventanas deslizantes.

2. Metodología

La primera parte del proceso fue de manera secuencial, sin hacer ningún cálculo, ya que estos se hicieron en la parte paralela.

2.1. Fase Secuencial

Esta primera fase la llevo a cabo un único hilo (el hilo maestro), el cual hizo lo siguiente:

1. **Lectura de Datos:** Se leyó el archivo `data01c1.csv` (con una frecuencia de muestreo de 100 Hz) de forma secuencial, y se almacena la serie temporal completa en un `std::vector`.
2. **Construcción de Ventanas:** Las ventanas se constuyeron con base en lo siguiente:
 - **Duración:** 2 segundos, lo que equivale a $(100 \text{ Hz} \times 2 \text{ s}) = 200$ muestras (N_w).
 - **Stride:** 50 %, lo que equivale a un deslizamiento de 100 muestras.

2.2. Fase Paralela: Cómputo de Métricas

Aquí es donde se utilizó **OpenMP** para paralelizar lo siguiente:

1. Se creó un "buffer de salida" del mismo tamaño que el número total de ventanas, para almacenar los resultados.
2. Se utilizó `#pragma omp parallel for` para repartir el bucle `for` (que itera sobre todas las ventanas) entre los hilos disponibles.
3. Cada hilo ejecutó la función `calcular_metricas` para sus ventanas asignadas, calculando las siguientes métricas por cada eje:
 - Media (μ) y Desviación Estándar (σ).
 - $RMS = \sqrt{\frac{1}{N_w} \sum x[n]^2}$.
 - Energía de la magnitud del acelerómetro ($E_{||a||}$), con las fórmulas: $||a|| = \sqrt{a_x^2 + a_y^2 + a_z^2}$ y $E_{||a||} = \sum_{n=1}^{N_w} ||a[n]||^2$.
4. Cada hilo escribió sus resultados directamente en el buffer compartido.

3. Verificación y Resultados

3.1. Consistencia del Cómputo

Para verificar que la paralelización no introdujo errores, se ejecutó el programa con diferente número de hilos y se comprobó que los resultados numéricos de las métricas son idénticos en todos los casos, por lo que los resultados no se ven afectados por el número de hilos utilizados. La Tabla 1 muestra la media por eje y la energía que resultaron de la ventana 1.

Cuadro 1: Verificación de consistencia del cómputo para la Ventana 0.

Nº Hilos	ax_media	ay_media	az_media	Energía ($E_{ a }$)
1 Hilo	0,9798	-0,0452	0,0286	192,6557
2 Hilos	0,9798	-0,0452	0,0286	192,6557
3 Hilos	0,9798	-0,0452	0,0286	192,6557
4 Hilos	0,9798	-0,0452	0,0286	192,6557
5 Hilos	0,9798	-0,0452	0,0286	192,6557
6 Hilos	0,9798	-0,0452	0,0286	192,6557
7 Hilos	0,9798	-0,0452	0,0286	192,6557
8 Hilos	0,9798	-0,0452	0,0286	192,6557

3.2. Resultados de Métricas

El programa procesó un total de 299 ventanas. La Tabla 2 muestra los resultados calculados para la primera ventana.

Cuadro 2: Métricas calculadas para la Ventana 1.

Eje	Media (μ)	Desv. Estándar (σ)	RMS
ax	0,9798	0,0115	0,9798
ay	-0,0452	0,0116	0,0467
az	0,0286	0,0140	0,0318
Energía ($E_{ a }$): 192,6557			

4. Análisis de Desempeño

Se comparó el tiempo de ejecución del cómputo de la versión paralela (usando 8 hilos) contra una versión secuencial del programa.

4.1. Tiempos Obtenidos

Los tiempos de ejecución medidos fueron:

- **Tiempo Secuencial (T_s):** 0.005886 segundos.
- **Tiempo Paralelo (T_p) (N=8):** 0.00300002 segundos.

4.2. Speed Up (SU)

La ganancia en velocidad (Speed Up) obtenida fue:

$$SU = \frac{T_s}{T_p} = \frac{0,005886}{0,003000} = 1,9620$$

La versión paralela fue 1.96 veces más rápida que la secuencial.

4.3. Eficiencia Computacional (E)

La eficiencia, que mide el aprovechamiento de los 4 núcleos, fue:

$$E = \frac{SU}{N} = \frac{1,9620}{8} = 0,2452$$

Se obtuvo una eficiencia del 24.5 %.

5. Conclusión

La versión paralela del cómputo fue más rápida que la versión secuencial, en específico, 1.96 veces más rápida, con una eficiencia computacional del 24.5 %. Además, los cálculos fueron los mismos independientemente del número de hilos utilizados, por lo que el programa utilizado es confiable.

Finalmente, el análisis de IMU's mediante paralelización resulta fundamental si se quiere aplicar otro tipo de métrica más robusta (no solo media o desviación estándar), como una Transformada de Fourier. Además, puede que existan casos en que los datos se hayan adquirido por varias horas, incrementando el tiempo de cálculo de las métricas. Esto hace que el método de paralelización se vuelva indispensable.

6. Anexo

Se presenta una captura de pantalla (Figura 1) de los resultados mostrados en la terminal al ejecutar el programa.

```
--- Lectura de CSV completada (SECUENCIAL) ---
Total de muestras leídas: 30000

--- Construyendo ventanas (SECUENCIAL) ---
Parametros: Tamano=200 muestras (2s), Stride=100 muestras (1s)

--- Verificacion de ventanas ---
Total de ventanas creadas: 299

--- Iniciando Paso 3: Calculo de Metricas (Paralelo) ---
Usando 8 hilos...
Calculo paralelo completado.

--- Verificacion de resultados ---
Tiempo total (paralelo): 0.00399995 segundos

--- Resultados de las primeras 3 ventanas ---
--- Ventana 0 ---
ax (Media/Std/RMS):  0.9795 /  0.0116 /  0.9796
ay (Media/Std/RMS): -0.0420 /  0.0119 /  0.0437
az (Media/Std/RMS):  0.0276 /  0.0113 /  0.0298
Energia Accel (E_||a||): 192.4820
--- Ventana 1 ---
ax (Media/Std/RMS):  0.9798 /  0.0115 /  0.9798
ay (Media/Std/RMS): -0.0452 /  0.0116 /  0.0467
az (Media/Std/RMS):  0.0286 /  0.0140 /  0.0318
Energia Accel (E_||a||): 192.6557
--- Ventana 2 ---
ax (Media/Std/RMS):  0.9798 /  0.0115 /  0.9798
ay (Media/Std/RMS): -0.0516 /  0.0172 /  0.0543
az (Media/Std/RMS):  0.0331 /  0.0211 /  0.0392
Energia Accel (E_||a||): 192.9131
```

Figura 1: En terminal se mostró el tamaño de la muestra, los parámetros para construir las ventanas, total de ventanas, el número de hilos utilizados, tiempo de ejecución, y las métricas calculadas por cada ventana.